FPGA HardCore Single Processor Implementation of RT Control Applications

Slim BEN OTHMAN, Meftah GHRISSI, Ahmed Karim BEN SALEM, Slim BEN SAOUD
L.E.C.A.P. - E.P.T. / I.N.S.A.T.
B.P. 676, 1080 Tunis Cedex, Tunisia
Slim.BenOthman@ept.rnu.tn, Meftah.Ghrissi@sotufem.com.tn, Ahmed.BenSalem@ept.rnu.tn, SlimBenSaoud@fulbrightweb.org

Abstract— **In recent years, Field Programmable Gate Array (FPGA) have proven themselves capable of handling a wide variety of tasks, from relatively simple electrical system control functions to more complex, algorithmic operations. According to the controller complexity, the FPGA design can be achieved by mixed Software/Hardware solutions. However, in many cases, the FPGA-based controller architectures require extensive Real Time (RT) testing before applying in real conditions. Thus, this paper will propose new approach to test controller design by implementing real time motor emulator linked to its controller drive in the same System on Chip (SoC). The gain with using this new approach is the ability to push past the operational limits of a specific environment and to test fault conditions that would otherwise be damaging or dangerous for a real motor. The proposed work describes an FPGA HardCore single processing implementation of a RT motor control loop and then explores the viability and cost-performance of this FPGA design.**

I. INTRODUCTION

Model-based design techniques have shortened the design cycle for engineers by leveraging software simulation and off-the-shelf hardware to test and validate electrical machine control algorithms [1].

Hardware-in-the-loop (HIL) simulation [2] takes the dynamics of electrical machine environments and models them using software to test newly designed controller performance. Corrective changes to the controller can be implemented and tested without fear of destroying expensive equipment. This design and co-simulation approach has been discussed in previous articles using Matlab Simulink and Xilinx System generator [3][4][5].

The disadvantage of this solution is its slower simulation times and its important material resources request. However, the hardware motor emulation concept that is firstly introduced by [6], is approved as a promising solution. Indeed, as presented in [7], a dSPACE hardware target is used to emulate inverter and motor when the control algorithms run in Field Programmable Gate Array (FPGA) circuit and control the completely emulated electric drive system. Yet, others interesting works, use the concept of emulation to only reproduce partial functions of the whole controller device. As in [1], authors depict the FPGA-based sensor emulators to allow designers to incorporate Real Time (RT) signals as early as possible in the design process. When in [8], paper gives a power system co-simulation case by using an FPGA-based emulator for series multicell converters.

FPGA technology is now considered by an increasing number of designers in various fields of application such as wired and wireless telecommunications, image and signal processing, medical equipment, robotics, automotive, and space and aircraft embedded control systems. Finally, industrial electrical control systems are also of great interest because of the ever-increasing level of expected performance while at the same time reducing the cost of the control systems [9].

Digital controllers implementation for power systems are nowadays more and more complex since they integrate sophisticated control algorithms and many additional tasks such as diagnosis and fault adaptive online.

Therefore, their RT implementation becomes a difficult procedure, which is usually performed using DSP processors associated with ASIC components. However, the use of FPGA circuits in this case represents a good compromise between the flexibility of a programming solution and the efficiency of a specific architecture with a high-integrated density.

Considering the benefits of FPGA implementation on electrical control design, this paper, will propose new approach to test controller design by implementing real time motor emulator linked to its controller drive in the same System on Chip (SoC). The FPGA HardCore processor will be achieved for this design. Therefore, the implementation concept, will be used either for complete controller device validation at the designing and validation stage or for diagnosis at normal functioning stage [6]. For these cases, the motor emulator should behave like the physical system functioning.

FPGA-based computing platforms, particularly those with embedded Hard or Soft microprocessors, have the potential to implement extreme high-performance applications without the up-front risk of creating custom fixed function hardware [10].

One of the often-overlooked benefits of using embedded processors within FPGA is the ability to create a hardware/software development target within a single programmable device. Even if the end-product will not include an embedded processor the ability to quickly download and test new software/hardware configurations can dramatically boost design productivity. Thus, by using the FPGA embedded processor as a full electrical machine system control loop can quickly verify RT behavior controller devices without proceeding to a physical test setup.

The paper is organized as follow: it firstly define the emulation concept, and present its contribution to validate controller design. Then, it specifies the model structure and

978-1-4244-1577-9/08/$25.00 ©2008 IEEE

constraints of electrical machine control drive and describe the implementation target platform. After that, it gives an overview, requirements, and underlying environment of our design, describing our system hardware and software architecture according to the application complexity and constraints. At the end, it shows the experimental result and it discusses the effect of single processor implementation on real time execution.

II. EMULATION CONCEPT

The emulation concept in the electric drive applications consist of designing an electronic system, which can reproduce the physical system functioning in real time and with high precision [6]. This system, called emulator, is used for the diagnosis and fault detection, as well as for the sensorless control devices (state observer).

Emulator can also contribute in the post-validation of embedded control systems. Then, before using the control device on the physical system and in order to avoid any design surprise (usually destructive and expensive); the control device is validated on the emulation step where it is connected to the emulator. This emulator should behave exactly like the physical system, in sense to make the control device believe that it is connected to the real CMS (Converter/Motor/Sensor) system. Thereby it has to generate and receive information similarly to the physical process: it receives control signals and generates information about the system state in forms identical to those obtained by sensors. After this step, the control device is completely validated and can be switched for use with the physical process as shown as Fig.1.

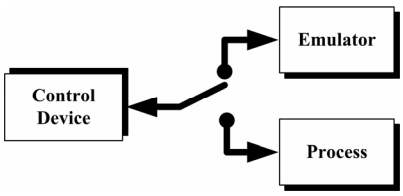

Figure 1. Structure of emulation application

Thus, to obtain precise results, we should use a precise models of different components used on this process and a high performance computing system with the lowest computing step as possible.

By taking advantage of SoC embedded processors, the new FPGA provides performance and flexibility to simultaneously simulate a variety of CMS emulators in real-time.

III. COMMAND DEVICE SPECIFICATION

A. *Electrical drive structure*

The electrical machine control is performed following the diagram of Fig. 2. Such a system is composed of two main parts: the process to control (CMS) and the control unit.

The control unit can be developed on an embedded processor. To validate this unit with minimal risk, we use the emulation concept. Indeed, we propose in this work, to implement the whole system on the SoC and to validate in real time the HIL functioning.

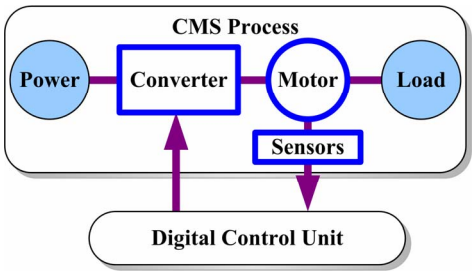

Figure 2. *Electrical drive structure*

Our case study is a physical process composed by a loaded DC motor supplied with a DC chopper. The control unit is composed by a PI current loop (fast loop) in cascade with PI speed loop (slow loop) (Fig. 3).

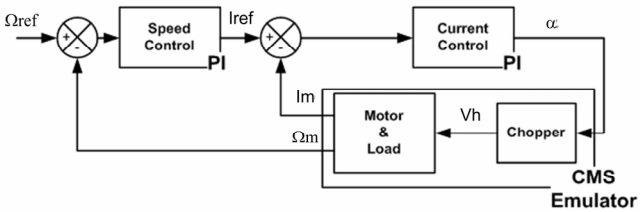

Figure 3. General diagram of motor control drive.

- PI Control: two PI loops was used to control interactive variables. The rotor speed (speed controller) and rotor torque (current controller) was both controlled by a separate PI module with physical limitation. The implementation is conventional [11] and is illustrated in Fig. 4.

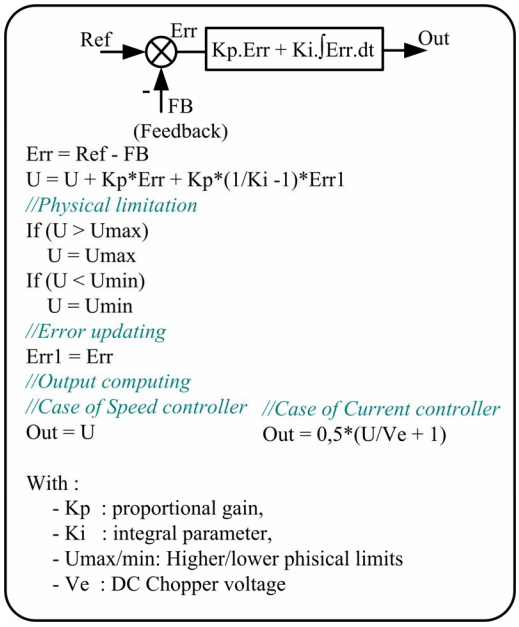

Figure 4. *PI Control specification.*

- Emulator: the motor emulator represents the DC motor with its DC chopper model. The implementation is illustrated in Fig. 5.

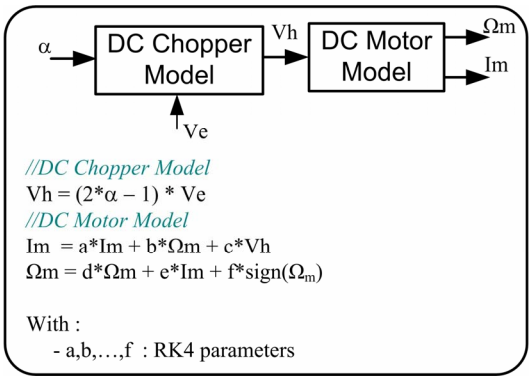

Figure 5. Emulator specification.

Although this process is relatively simple, it is a fast way to validate emulation concept and to determine the controller SoC implementation limits.

B. *Electrical drive constraints*

The digital motor model is approximated by 4th order Runge-Kutta method (RK4) as shown as Fig.5. This model, must compute cyclically the rotation speed Ωm and the current Im according to the pulse width α. It is constrained by computed step h that should be as short as possible to have a well-approximated model for the real motor.

More over, the main constraints of control devices are time execution and arithmetic precision [6].

In our case, speed and current controller algorithms, are constrained by the following specifications:

- a period Tc = 550µs for the current control loop;
- a period Tm = 40ms for the speed control loop;

In order to respect all these timing constraints, we must use an appropriate implementation target as well as an appropriate hardware and software architecture.

IV. TARGET PLATFORM AND CORE SELECTION

The selected development platform consists of Xilinx ML310 board [12]. It provides a Multiprocessor SoC (MPSoC) FPGA. Its Xilinx Virtex-II Pro XC2VP30 FPGA combines more than 30,000 configurable logic blocks (CLB) and 2 PowerPC405 HardCores [13]. The large amount of peripherals offers a variety of different interfaces.

The Xilinx Embedded Development Kit (EDK) environment provides the tools and libraries to integrate the IBM PowerPC 405 [14] cores on chip, soft MicroBlaze cores, IBM CoreConnect buses, and customizable peripherals to design multiprocessor micro-architectures [15].

V. IMPLEMENTATION

A. *Hardware overview*

The embedded processor system contained in this example design consists of PowerPC 405 (PPC405). It is a HardCore processor based on the IBM PowerPC processor family and is capable of achieving 400 MHz and 600+ DMIPS performance. It is a RISC core (32-bit Harvard architecture) with large, 16 KB, 2-way set-associative instruction, and data caches. It supports enhanced instruction and data On-Chip Memory (OCM) controllers that interface directly to embedded Block RAM (BRAM) memory. The PowerPC 405 supports the IBM CoreConnect bus architecture [16] and has high performance debug and trace features.

The hardware system design is shown in Fig. 6. PowerPC processor use Processor Local Bus (PLB) as the main system interface. Slow peripherals such as timer, UART and interrupt controller are available with the On-chip Peripheral Bus (OPB) interface, and are accessible via a PLB to OPB bridge peripheral.

UART peripheral is an RS232 serial connection that allows communication with a PC. General Purpose Input Output (GPIO) peripheral is used to interface external logic analyzer to PowerPC.

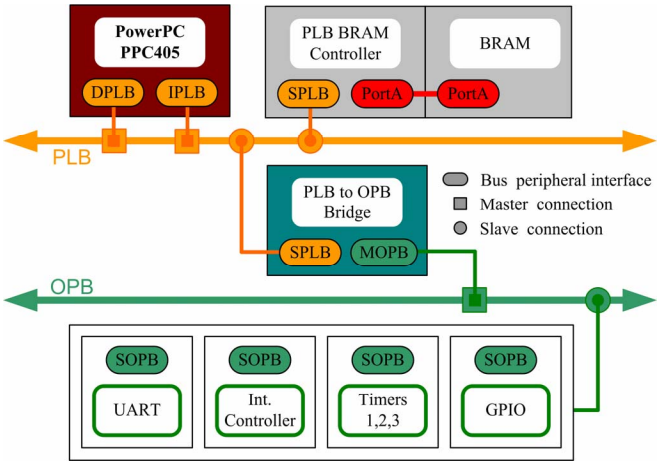

Figure 6. PowerPC hardware block diagram

The interrupt controller peripheral [14] was used for handling the timer interrupts. The software controller interrupt handler ensures that individual interrupt handlers in the order of their priorities handle interrupts from the peripherals. The interrupt signal output of the controller was connected to the non-critical interrupt input of the PowerPC. Fig. 7 depicts timer interrupt signals with priorities 1 through 3 connected to the interrupt controller input.

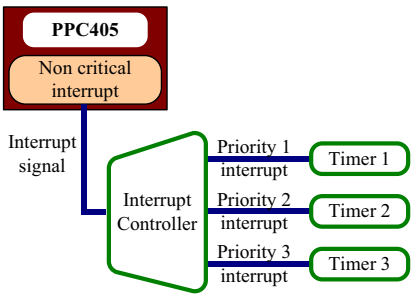

Figure 7. Interrupt controller and timer peripherals

B. *Software design*

Traditional software systems employed a looping construct for the main part of the application and used interrupts to

handle time-critical events [17]. We have considered in this work, two partitioning approaches of the software handler routines:

- In the first one, we have used two interrupt routines: emulator and controller. The current controller is executed on each invocation of the interrupt handler and the speed controller is executed on every 70th invocation.
- In the second one, the controller routine has been divided in current and speed controller routines. We have used therefore three independent interrupt routines.

With these both approach, software routines has been configured with an interval timer using a real time interrupt clock as a peripheral device. The timer allows task control on a timed basis [18].

VI. CONTROL UNITS VALIDATION

The FPGA implemented code runs all handler interrupt routines and displays the computed results using the UART. We have stored all current and speed responses for different speed targets. The plotted figures are promising and show that implemented control loop is well functioning.

In the following Fig. 8, we have illustrated an example of responses for the two architectures with a 100rad/s speed target and a 600µs computing step (equal to a Tc current sampling time).

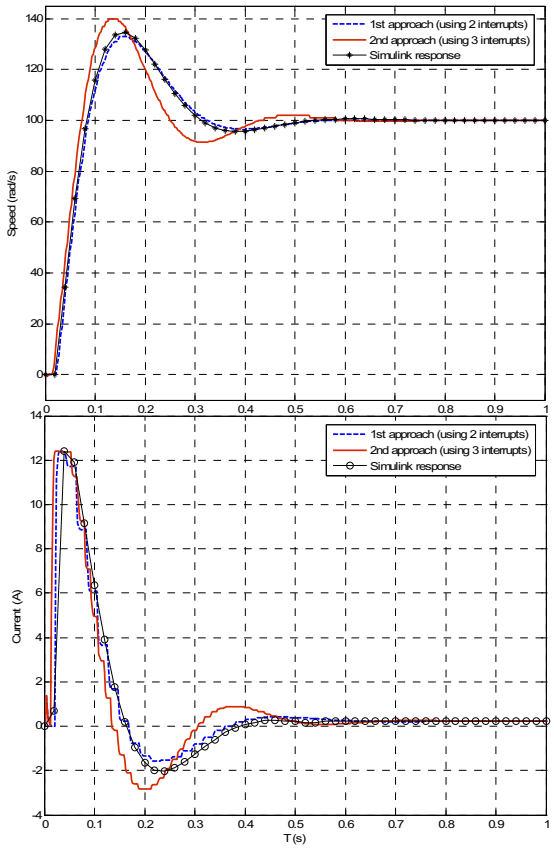

Figure 8. Speed and current responses with speed target of 100rad/s.

The last curves show clearly that the speed reached exactly the fixed reference speed 100rad/s. In addition, we can see plainly the similarity between the values generated by the Simulink simulation and our first design.

However, the dynamic response of the second implemented approach is better than the first one. Besides, the obtained results with the FPGA implementation depend a lot on the choice of the emulator computing step *h* that must be enough high. This is in opposition with the emulator concept that needs to run on short time step to be closer to the real physical process.

This timing problem could be linked to the interrupts management. To probe this problem, we proceed to a time analyze of execution routines.

VII. TIMING ANALYSIS

The best means to analyze execution times of interrupt routines as well as the controller interrupts management is the use of a logic analyzer [19]. In this case, we tied the analyzer to signals driven by the PowerPC processor using the GPIO IP (Fig. 9).

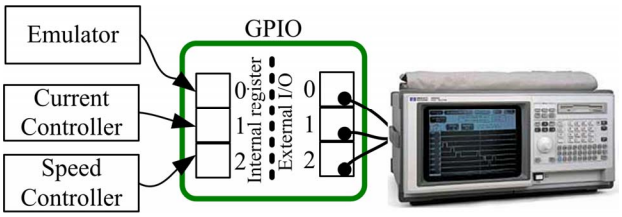

Figure 9. Sw timing analysis with logic analyzer.

When an interrupt occur, the corresponding handler routine starts with setting to high its reserved bit in GPIO register and finish with switching it to low. Through this method, we have determined the processing time of every function (Table. 1.).

TABLE I. FUNCTION DELAYS.

Functions	Processing time (us)
Emulator	204
Current controller	123
Speed controller	82

It is obvious that the emulator computing time, in this case, is important and can conduct to imprecision problems on the model. To use emulator in high-level priority with a computing step lower to 204µs implies an interrupt overload and an important delay on the controller routines.

Considering the second software architecture (3 interruptions) with *h*=600us, the interrupt routines turn correctly as shown in Fig. 10. The timer interruptions are well managed by the processor and the timer controller.

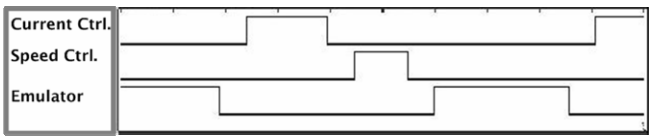

Figure 10. 2nd Sw architecture interrupt management with *h*=600µs.

Although when *h*=300us (Fig. 11) we have observed a defect of interrupt routines execution. The speed loop never has time to run. The current loop is regularly interrupted and is not work adequately.

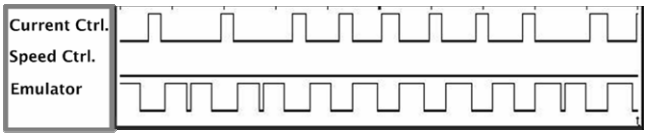

Figure 11. 2nd Sw architecture interrupt management with *h*=100µs.

Even while using 1st architecture, the control unit execution delay become variable and sometimes conducted to an interrupt overload (Fig. 12).

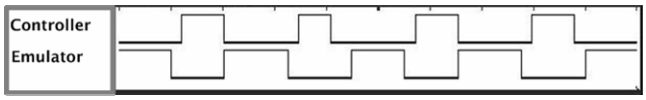

Figure 12. 1st Sw architecture interrupt management with *h*=300µs.

Due to the high processing time of the emulator interrupt routine, the current and speed interrupts are inhibited until execution is complete. When the interruption time occurrence of the emulator routine is decreased, the latency of the other interrupts increases and then these interrupts will never have enough time to wholly execute the corresponding routine instructions code. For these reasons, the minimal period of emulator must be computed according to the sum of all routines processing time and the interrupt latencies. In our case, it is about 350us.

VIII. Conclusion

At the end of this work, we have succeeded to implement a new validation approach of a motor drive control device while using emulation concept. However, the presented design models a real system but it does not support sensor functionalities, PWM generation, and instantaneous order tension computation.

In addition, we have discussed several difficulties to the level of emulation routine latency and interrupt management (timing, partitioning, priorities ...). To remedy all these problems, we propose in the future work, distribution of interruptions on several processors and use of hardware implementations of some control loop devices.

Acknowledgments

The authors are grateful to Xilinx Inc. for donations, which made possible the research on some aspects of full digital controller implementation.

References

[1] V. Aggarwal and R. Kuhlman, "Simulating Sensors on FPGA Hardware," *Embedded Systems Conferences*, ESC-402, 2007.

[2] H. Shouling and X. Xuping, "Hardware simulation of an adaptive control algorithm," *in Proc. of the 18th IASTED, Modeling and simulating*, MR, Canada, Mai-Jun. 2007, pp. 102-107.

[3] A. K. Ben Salem, S. Ben Othman and S. Ben Saoud, "Design and implementation of a neural command rule on a FPGA circuit," *in Proc. IEEE ICECS*, Gammarth, Tunisia, Oct. 2005.

[4] H. Abdelkrim, S. Ben Othman and S. Ben Saoud, "Hardware implementation of fuzzy logic controller," *in Proc. IEEE ICECS*, Gammarth, Tunisia, Oct. 2005.

[5] S. Ben Othman and S. Ben Saoud, "Design and hardware implementation of embedded controllers devices," *in Proc. ICMNT*, Tizi-Ouzou, Algeria, Nov. 2006, pp. 75-76.

[6] S. B. Saoud, "Emulateur temps réel d'associations Convertisseurs statiques / Machines électriques / Capteurs : Etude, conception et réalisation," *PHD thesis, ENSEEIHT*, INPT, France,1996.

[7] O. Laakkonen, K. Rauma, H. Saren, J. Luukko and O. Pyrhonen, "Electric drive emulator using dSPACE real time platform for VHDL verification," *in proc. IEEE MWSCAS*, Vol. 3, Jul. 2004, pp. 279-282.

[8] R. Ruelland, G. Gateau, T. A. Meynard and J.C. Hapiot, "Design of FPGA-based emulator for series multicell converters using co-simulation tools," *IEEE Trans. Power Electron.*, Vol. 18, no. 1, Jan 2003, pp. 455–463.

[9] E. Monmasson and M. N. Cirstea, "FPGA design methodology for industrial control systems—A review", *IEEE Trans. Industrial Electronics*, Vol. 54, No. 4, pp. 1907-1925, Aug.2007.

[10] D. Pellerin, J. Smith and S. Thibault, *Mixed FPGA Processor Platforms Accelerate Software Algorithms*, RTC magazine of record for the embedded computing industry, Jan. 2004.

[11] J. Zambada, *Sensorless Field Oriented Control of PMSM Motors*. Microship Technology Inc, DS01078A, 2007.

[12] Xilinx, *ML310 User Guide, VirtexII pro Embedded Development Platform*. UG068, Jan. 2005.

[13] Xilinx, *Virtex II pro Platform FPGA, Complete Data Sheet*. DS083, Oct. 2005.

[14] Xilinx, *PowerPC 405 Processor Block Reference Guide, Embedded Development Guide*. UG018, Jul. 2005.

[15] V. Asokan, *Designing Multiprocessor Systems in Platform Studio*, XPS White Paper, WP262, Apr. 2007.

[16] IBM, *The CoreConnect Bus Architechture*. IBM Corporation, 2001.

[17] B. H. Fletcher, "FPGA Embedded Processors," *Embedded Systems Conference*, ETP-367, San Francisco, 2005.

[18] J.Regehr, U. Duongsaa, "Preventing Interrupt Overload in Embedded Systems," *in Proc. ACM LCTES*, USA, Jun 2005.

[19] B. Frieden, *FPGA Embedded Test Speeds Development*. Xilinx Embedded magazine, issue 4, pp. 46-51, Nov. 2006.